Practical Lab 02: CPU Utilization and Process Monitoring

Objective: Observe your eight logical CPUs, examine process activity, and understand how Linux reports CPU workload utilization.

We will run this lab step by step on your Ubuntu WSL2 installation.

Step 1 — Inspect your processor topology

Open Ubuntu and execute:

```
lscpu | grep -E '^(CPU\(s\)|Model name|Thread\(s\) per core|Core\(s\) per socket|Socket\(s\))'
```

This filters lscpu output to show the most relevant CPU specifications.

Next, run:

```
nproc
```

Step 2 — Monitor real-time CPU activity

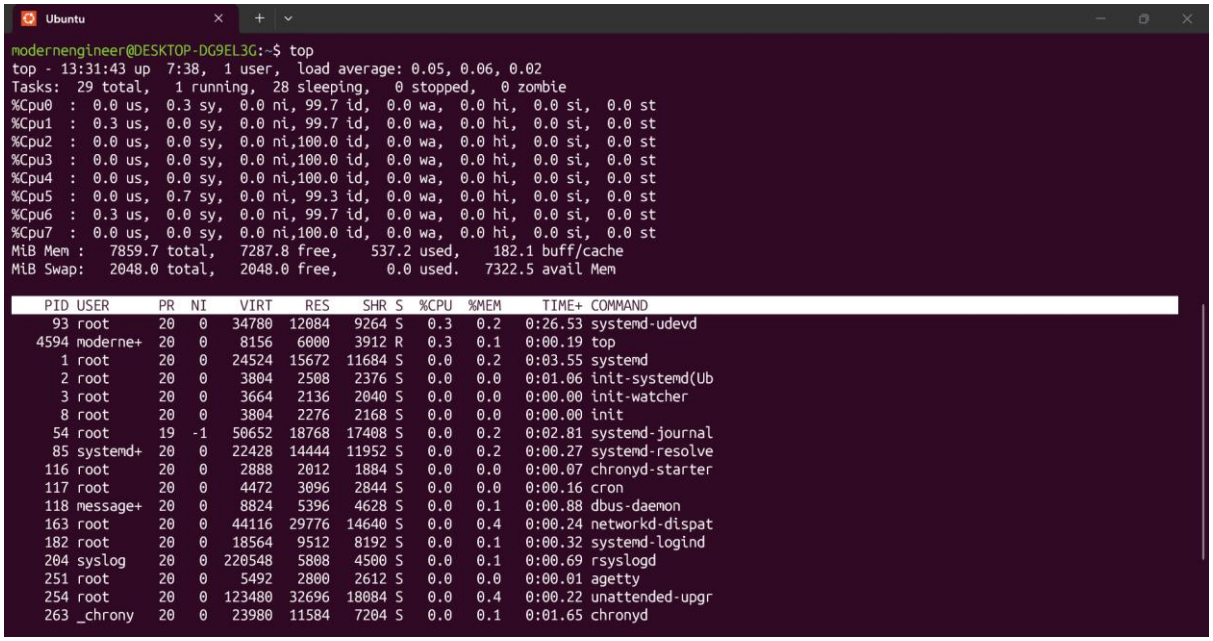
Execute:

```
top
```

This launches the Linux process-monitoring utility.

While top is running:

1. Press 1 to display individual logical CPU utilization, where supported.
2. Observe %Cpu0 through %Cpu7, or similarly named CPU lines.
3. Examine the %CPU column for processes.
4. Press q to exit.



Let’s analyse the results:

Metric	Observed value	Interpretation
Logical CPUs	8	All eight are visible
CPU idle	99.7–100%	Almost no CPU activity
Running tasks	1	One task actively running
Sleeping tasks	28	Processes waiting for work
Load average	0.05, 0.06, 0.02	Very little system demand
Total tasks	29	Processes visible to top
Memory total	7,859.7 MiB	Approximately 7.7 GiB
Memory available	7,322.5 MiB	Approximately 7.15 GiB
Swap used	0 MiB	No swap usage during the test

CPU utilization: 8 logical processors are almost entirely idle. The system has substantial unused CPU execution capacity at the time of measurement.

Load average: The values of 0.05, 0.06, and 0.02 indicate very little runnable or uninterruptible-sleep task demand during the respective averaging periods.

Processes: 29 visible tasks, but only one is actively running at the instant shown. The others are sleeping, which is normal.

Memory: Ubuntu has approximately 7.15 GiB of memory available, so there is no obvious memory pressure in this snapshot.

Understanding the CPU columns

%Cpu0 : 0.3 us, 0.0 sy, 99.7 id

%Cpu1 : 0.3 us, 0.0 sy, 99.7 id

Here are the important fields:

Field	Meaning
us	Time executing user-space code
sy	Time executing kernel code
ni	Time executing processes with adjusted nice priority
id	CPU idle time
wa	Time accounted as I/O wait
hi	Hardware interrupt processing
si	Software interrupt processing
st	Steal time reported by virtualization

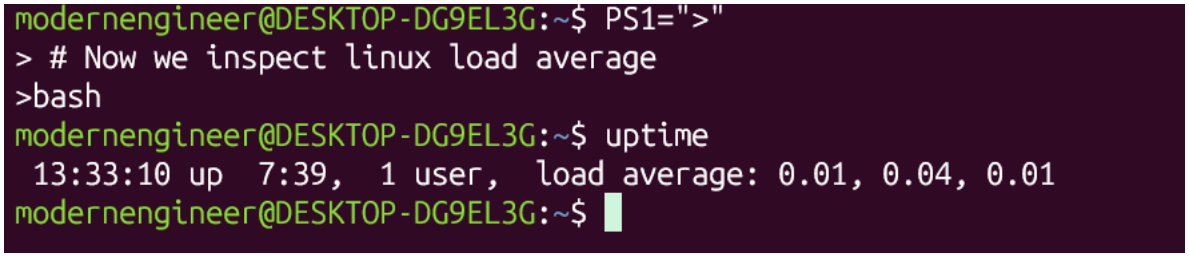
For example, 99.7 id means the logical CPU was idle for approximately 99.7% of the measurement interval.

These percentages describe CPU time, not the percentage of the CPU's maximum clock frequency.

Step 3 — Inspect Linux load average

Execute:

Uptime



Generate a Controlled CPU Workload

Now we will observe how CPU utilization changes when an actual process performs computation.

Objective: Run a lightweight, single-process CPU workload and monitor its effect on your eight logical processors.

Step 1 — Inspect CPU Topology

Execute:

lscpu

nproc

Observed Result:

The Linux environment reported eight logical processors.

Step 2 — Monitor CPU Activity

Execute:

top -d 1

Press 1 to display individual logical CPU utilization.

This allows CPU activity to be monitored for each logical processor.

Baseline Observations

Before running the workload:

- Individual CPU utilization was very low.
- Logical CPUs were approximately 99–100% idle.
- The system showed low load averages.
- No substantial memory pressure was observed.

Step 3 — Generate a Controlled CPU Workload

Open a second Ubuntu terminal and execute:

timeout 30s bash -c 'while :; do :; done'

This command executes a CPU-intensive Bash loop.

timeout 30s automatically terminates the workload after approximately 30 seconds.

The loop does not generate output, but it consumes CPU execution time.

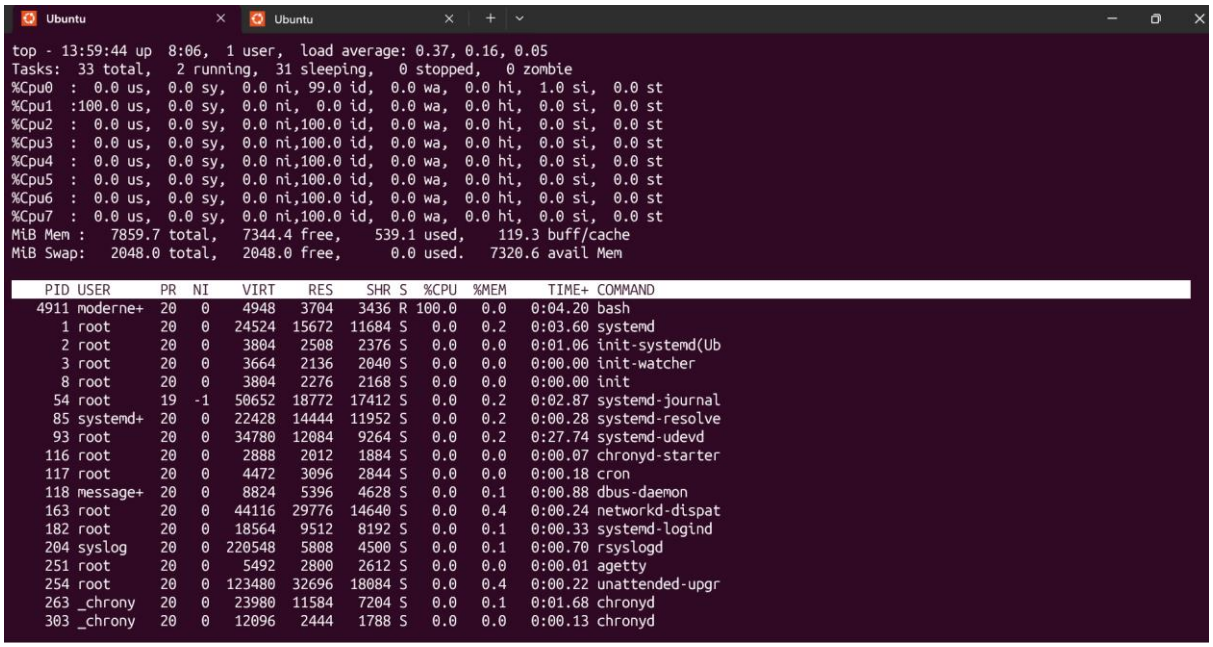
Watch Terminal 1

While the workload is active, look for a bash process in the process list.

You should expect:

- The bash process to consume substantial CPU time, potentially approaching 100% of one logical CPU.
- At least one logical CPU to show increased activity.
- Other logical CPUs to remain comparatively idle.
- CPU utilization to return toward baseline after 30 seconds.

The workload may migrate between logical processors, so the busy CPU number may change.



Understanding your experiment

timeout 30s bash -c 'while :; do :; done'

The command created a continuously executing Bash loop.

While it was running, top reported:

PID %CPU COMMAND

4894 100.0 bash

And one logical processor showed:

%Cpu4 : 100.0 us, 0.0 sy, 0.0 id

The experiment demonstrated that a CPU-intensive process can fully utilize one logical processor without saturating the entire processor.

The Bash workload reached approximately 100% process CPU utilization.

One logical processor, CPU4, showed 100% user-space utilization during the captured observation.

Other logical processors remained largely idle.

With eight logical processors, this corresponds to roughly 12.5% of total logical CPU execution capacity, assuming the other processors are idle.

The significance of these measurements is that the workload saturated one logical processor's execution time, not all eight logical processors.

Overall CPU utilization can remain relatively low even when one execution thread has reached its processing limit.

Why this matters for AI infrastructure

Consider an NVIDIA GPU server running an inference service.

The CPU handles supporting operations such as API processing, tokenization, request scheduling, and application logic.

If one critical CPU-side thread becomes saturated, it may limit how quickly requests can be prepared and delivered to the GPU.

The GPU could therefore have unused compute capacity even though the application experiences increased latency.

This is why production engineers examine per-core activity, process-level utilization, and application latency, rather than depending exclusively on aggregate CPU utilization.

Production Troubleshooting Scenario

Scenario

An AI inference server experiences:

- Increasing API response latency.
- Relatively low GPU utilization.
- Moderate overall CPU utilization.
- One heavily utilized CPU processing thread.

Investigation

An infrastructure engineer should inspect:

1. Per-core and per-thread CPU activity.
2. Request preprocessing and tokenization time.
3. CPU scheduling and resource limits.

4. Request queue latency.
5. GPU computation and utilization metrics.
6. Memory pressure and data-transfer overhead.